Time Series Analysis and Model Evaluation

Exploring EDA and Forecasting Techniques

Introduction

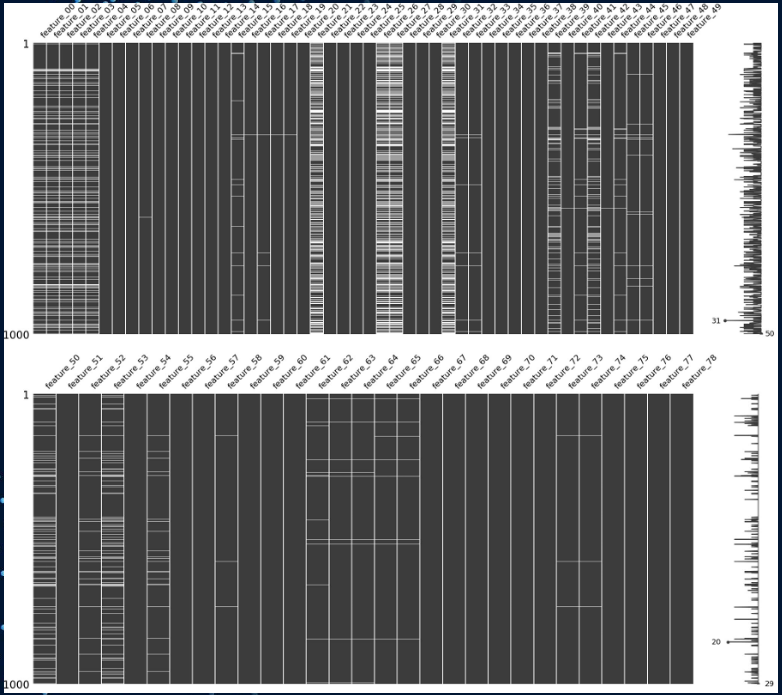
Welcome to our presentation by **BRUTE VOL.1**, where we explore key aspects of time series forecasting. This includes **exploratory data analysis (EDA), model selection**, and an in-depth evaluation of forecasting models such as **GRU, LSTM, and LightGBM**. Our analysis aims to uncover insights that drive more accurate and reliable financial predictions.

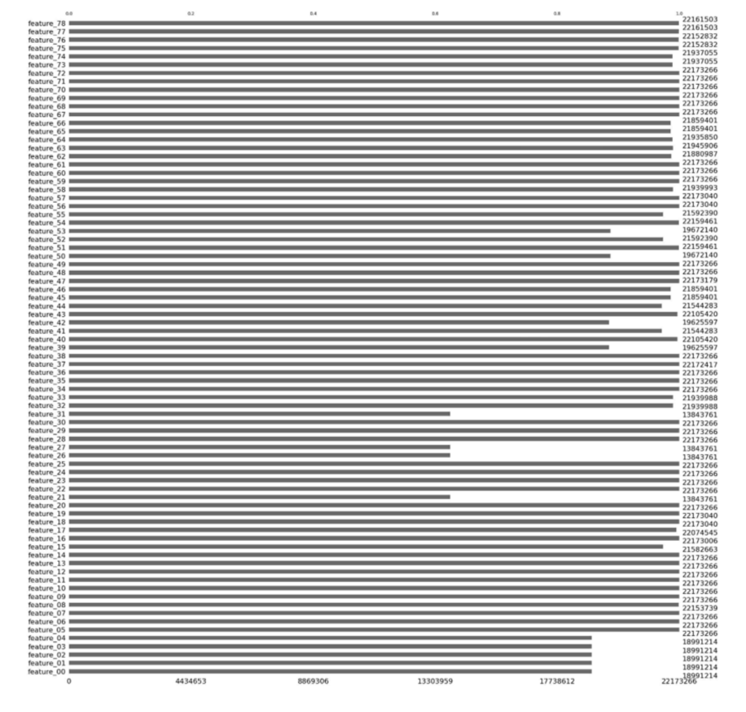
Table of contents

Exploratory Data Analysis (EDA)

- Handling Missing Values: Understanding the impact of missing data and strategies for imputation based on percentage of missing values.
- Distribution analysis: Applied suitable transformation to convert various distributions to normal distribution.
- Correlation Analysis: Utilizing heatmaps to visualize relationships between features, identifying patterns and multicollinearity.
- Target Analysis: Examining stock-specific trends, outliers, and variability through visualizations.

Missing Values

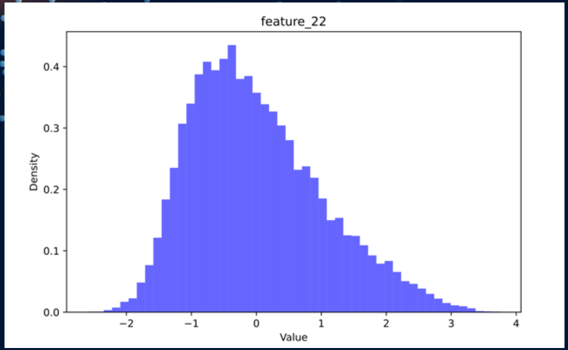




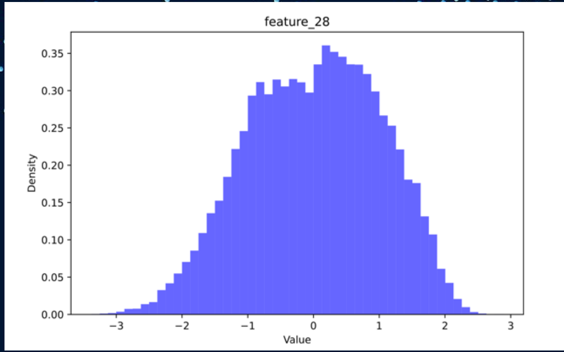
Feature Distributions

- **Normal Distribution**: feature 01, feature 02, feature 03, feature 04, feature 05, feature 06, feature 07, feature 08, feature 18, feature 19, feature 31, feature 32, feature 33, feature 34, feature 35, feature 36, feature 37, feature 38, feature 39, feature 40, feature 41, feature 42, feature 43, feature 44, feature 45, feature 49, feature 50, feature 51, feature 52, feature 53, feature 54, feature 55, feature 56, feature 60, feature 64, feature 65.
- **Log-Normal Distribution:** feature 12, feature 13, feature 14, feature 15, feature 16, feature 17, feature 30, feature 61, feature 62, feature 63.
- **Exponential Distribution:** feature 20, feature 21.
- **Gamma Distribution:** feature 22, feature 23, feature 24, feature 25, feature 66, feature 67, feature 68, feature 69, feature 70.
- **Beta Distribution: feature 09, feature 10.**
- **Uniform Distribution**: feature 26, feature 27, feature 29.
- **Gaussian Mixture Model (GMM):** feature 28, feature 47, feature 48, feature 57, feature 58, feature 59.
- **F Distribution:** feature 71, feature 72, feature 73, feature 74, feature 75, feature 76, feature 77.
- **Binomial Distribution:** feature 09, feature 10.
- **Poisson Distribution**: feature 11.

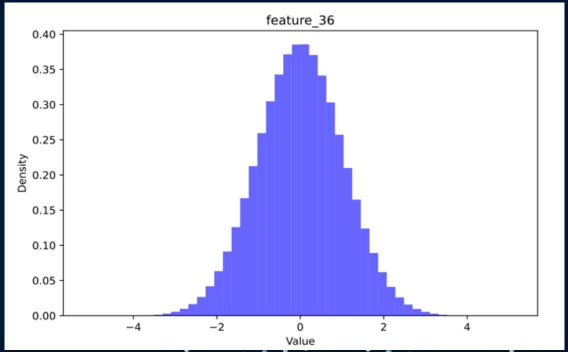
Distributions



Gamma Distribution



GMM

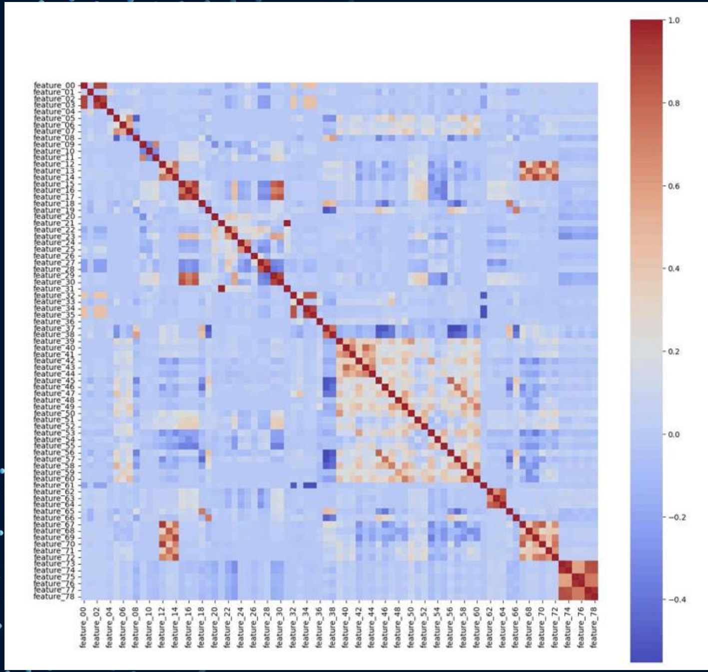


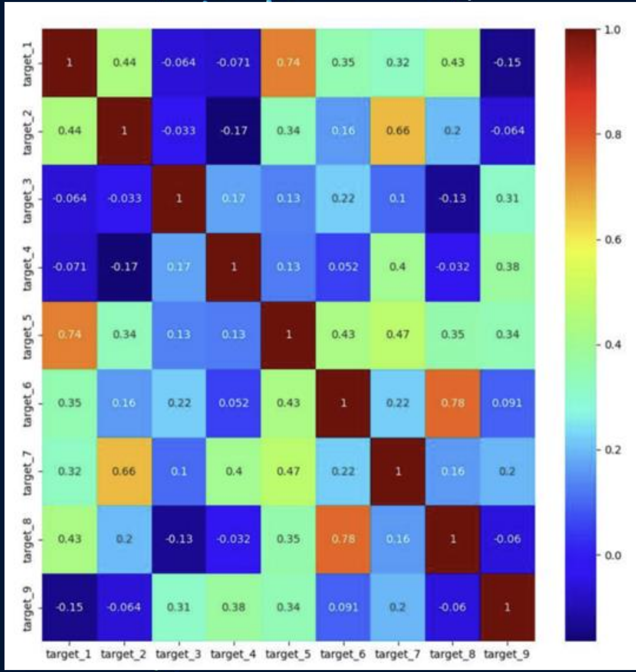
Normal Distribution

Transformations

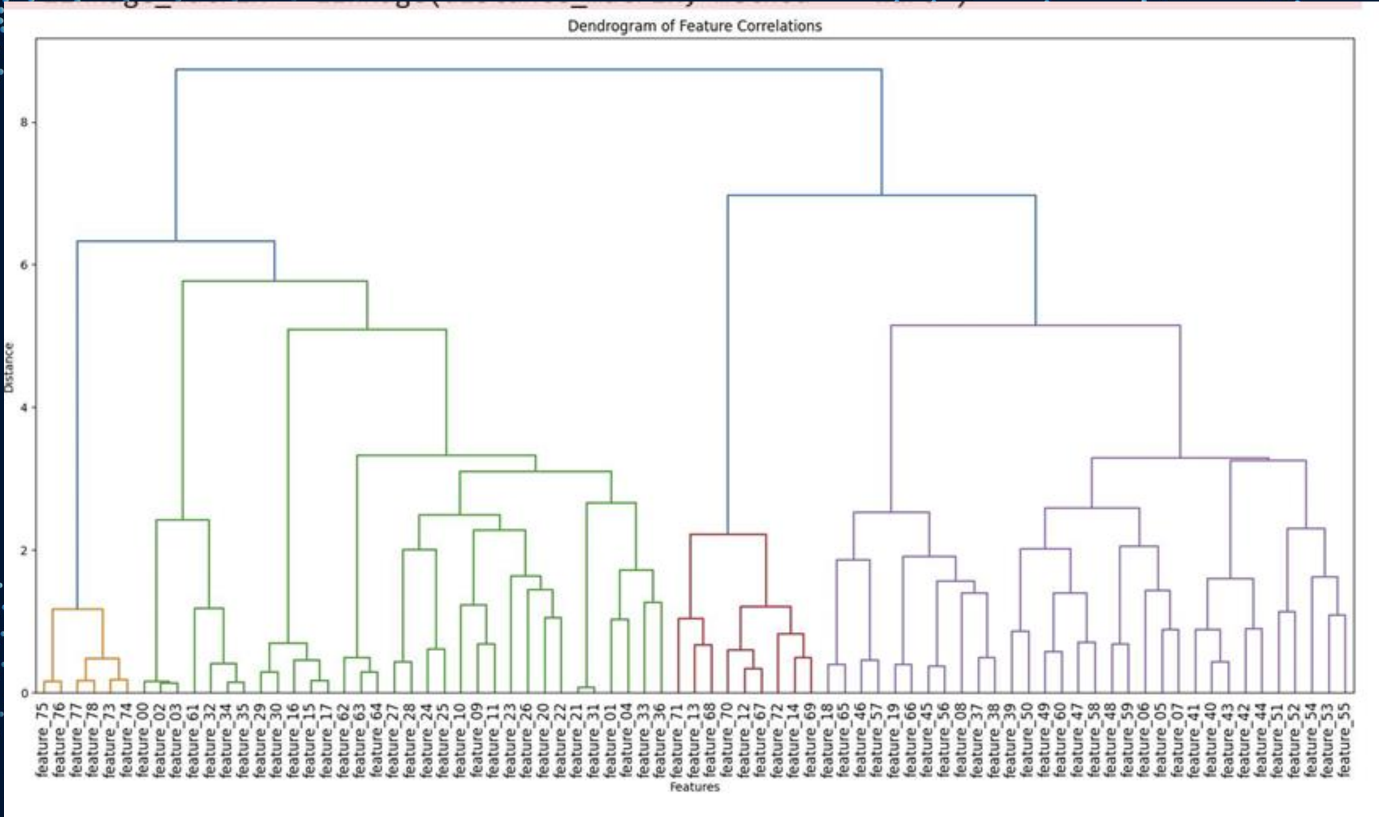
```
for col in LOGNORM + EXPON + CHI2 + GAMMA + F:
    all_df_imputed[col+'_log'] = np.log1p(all_df_imputed[col])

for col in GAMMA + BETA + LOGNORM:
    mini = all_df_imputed[col].min()
    if mini <= 0:
        all_df_imputed[col+'boxcox'] = PowerTransformer(method='box-cox').fit_transform(1-mini + all_df_imputed[col].values.reshape(-1, 1))
    else:
        all_df_imputed[col+'boxcox'] = PowerTransformer(method='box-cox').fit_transform(all_df_imputed[col].values.reshape(-1, 1))

for col in T:
    all_df_imputed[col+'_yeo_johnson'] = PowerTransformer(method='yeo-johnson').fit_transform(all_df_imputed[col].values.reshape(-1, 1))

for col in CHI2 + POISSON:
    all_df_imputed[col+'_sqrt'] = np.sqrt(all_df_imputed[col])

for col in UNIFORM:
    scaler = MinMaxScaler()
    all_df_imputed[col] = scaler.fit_transform(all_df_imputed[col].values.reshape(-1, 1))
```

Correlation Heatmaps

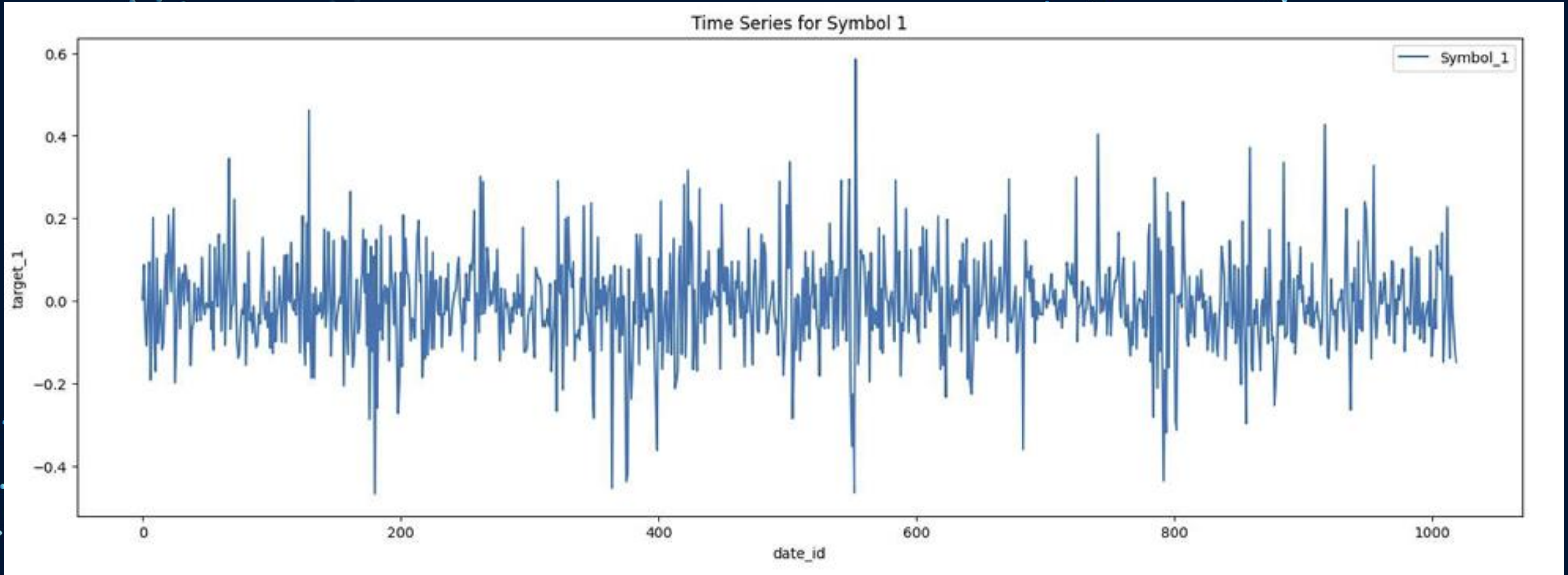




Correlation Dendograms



Each Symbol Time Series



Model Analysis

GRU Model: The GRU model efficiently captured long-term dependencies while maintaining computational efficiency, making it a suitable choice for sequential stock data.

LightGBM Model: Despite leveraging gradient boosting, it struggled with overfitting and produced negative R^2 values, highlighting challenges in feature selection and hyperparameter tuning.

LSTM Model: The LSTM model effectively handled complex temporal dependencies, and training modifications like reduced batch size and adjusted learning rate improved stability and convergence.

Why these models?

- **GRU Model**
  - Efficient for sequential data, mitigates vanishing gradients.
  - 500 hidden units for capturing long-term dependencies.
  - Gradual dropout prevents overfitting.
  - Adam optimizer ensures stable learning.
- **LSTM Model**
  - Effective for long-term dependencies in stock price prediction.
  - Two-layer stacked LSTM for hierarchical feature extraction.
  - Dropout (0.2) prevents overfitting, LeakyReLU avoids dead neurons.
- **LightGBM Model**
  - Fast, scalable tree-based model for tabular data.
  - Used GPU acceleration and parallel processing.

GRU Architecture

```
GRUModel(
  (gru): GRU(133, 500, batch_first=True)
  (dropout_gru): Dropout(p=0.3, inplace=False)
  (fc): Sequential(
    (0): Linear(in_features=500, out_features=500, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.2, inplace=False)
    (3): Linear(in_features=500, out_features=300, bias=True)
    (4): ReLU()
    (5): Dropout(p=0.1, inplace=False)
    (6): Linear(in_features=300, out_features=1, bias=True)
  )
)
```

GRU Epoch vs Loss

LSTM Architecture

```
StockLSTM(
  (lstm): LSTM(141, 128, num_layers=2, batch_first=True, dropout=0.2)
  (fc): Linear(in_features=128, out_features=1, bias=True)
  (relu): LeakyReLU(negative_slope=0.01)
)
```

LSTM (Learning Rate – 0.001)

LGBM

- n_estimators: 1000, learning_rate: 0.005
- min_child_samples: 100, num_leaves: 50
- subsample: 0.9, colsample_bytree: 0.9
- random_state: 42, device: GPU

Did not get satisfactory results

Performance Comparison

• **Evaluation Metrics:** We compared models using key metrics such as **R^2 score**, assessing their ability to explain variance in financial time series data.
• **Hyperparameter Tuning:** Optimization efforts, including learning rate adjustments and architecture modifications, significantly influenced model performance.
• **Model Suitability:** Insights from the results highlight the trade-offs between complexity and accuracy, guiding the selection of the most effective forecasting approach.

```python
class R2Loss(nn.Module):
    def __init__(self):
        super(R2Loss, self).__init__()

    def forward(self, y_pred, y_true, weights):
        ss_total = torch.sum(weights * (y_true ** 2), dim=0)
        ss_residual = torch.sum(weights * ((y_true - y_pred) ** 2), dim=0)
        r2 = 1 - (ss_residual / (ss_total + 1e-8))
        return 1 - torch.mean(r2)
```

What results did we get? (R2 scores)

- **LSTM (LR = 0.001): 0.0184** (Bestperforming)
- **GRU: 0.0079**
- **LSTM (LR = 0.0001): 0.0027**
- **LightGBM: Negative (underperformed)**

Key Features

- LightGBM struggled with high data complexity and possible overfitting.
- Lower learning rates improved stability but slowed convergence.
- Reducing batch size sped up training but introduced noise.

Summary and Deductions

- **LSTM outperformed GRU and LightGBM for financial time series prediction.**
- **Higher learning rates caused instability; lower rates improved convergence.**
- **LightGBM struggled due to feature complexity—tree models may not be optimal for this task.**
- **Future improvements:**
    - Feature engineering to enhance informative signals.
    - Fine-tuning GRU/LSTM hyperparameters further.
    - Exploring hybrid models combining deep learning and boosting techniques.

Conclusion

Our findings emphasize the significance of selecting the right model and fine-tuning parameters to enhance time series forecasting accuracy. By leveraging GRU, LSTM, and LightGBM, we explored different approaches to capturing trends and dependencies in financial data. Moving forward, continuous model refinement and feature engineering will be key to improving predictive performance.

Thank you!